Linux Process Monitoring & Control System

Team7 | Section: S13 |Members:Adithya Vishnubatla–[2520080036], AbhinavSunkara–[2520080043], T. Harsha Vardhan – [2520080047] | Guide: [Dr.D Alekhya]

Operating Systems and Systems Programming (OSSP)25CS2104E | KLH BachupallyCampus,CSE | PBL 2026–27

Abstract

- Linux process data is spread across /proc files, and top / ps can stay a black box for students. This C/C++ utility reads /proc directly to monitor and control processes with POSIX signals.

Outcome: a transparent tool showing PID, PPID, state, CPU, memory and command, plus validated control.

Problem Statement & Objectives

- **Problem:**Many processes run at once and their details are split across /proc/[pid]/stat,status and cmdline; users need clear usage, parent–child links and safe control.

Objective 1: Monitor PID, PPID, state, CPU and memory usage.
Objective 2: Control processes with validated signals via kill().
Objective 3: Provide a clear, safe Linux CLI for both.
Scope: Linux CLI on /proc and POSIX APIs; SIGTERM / SIGKILL on permitted processes.

Existing vs Proposed System

Aspect	Basic / ExistingApproach	Proposed Project
Process visibility	top / ps show data; internals stay a black box	Parses /proc files; shows PID, PPID, state, CPU, MEM
Data access	Details scattered across /proc files	One organized process table, refreshed periodically
Control	Viewing and acting are separate steps	Pick a PID, send permitted signals via kill()
Error handling	Hidden inside standard tools	Checks PID and permission; reports clear errors
Key gap	Data exists but is not clear or actionable	Transparent utility on Linux / POSIX APIs

Feasibility Study

- **Technical:** C/ C++ on Linux / Ubuntu using GCC / G++, POSIX / Linux APIs and the /proc filesystem.

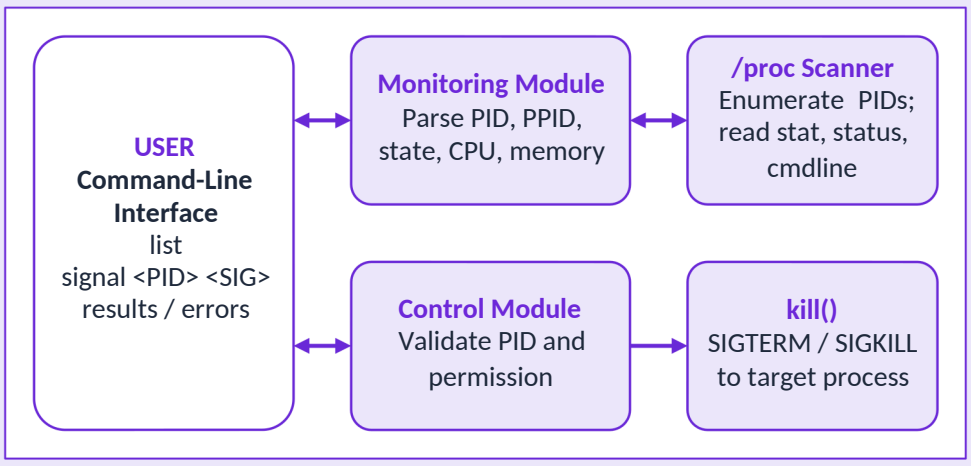
Economic: Open-source tools only (Linux, GCC / G++); no special hardware or hosting needed.
Operational: Simple CLI (list, signal); validates PID existence and permission before any signal is sent.
Schedule: Monitoring, control, UI / testing and documentation are developed by role and integrated within the Term-I timeline.

Tools & Technologies

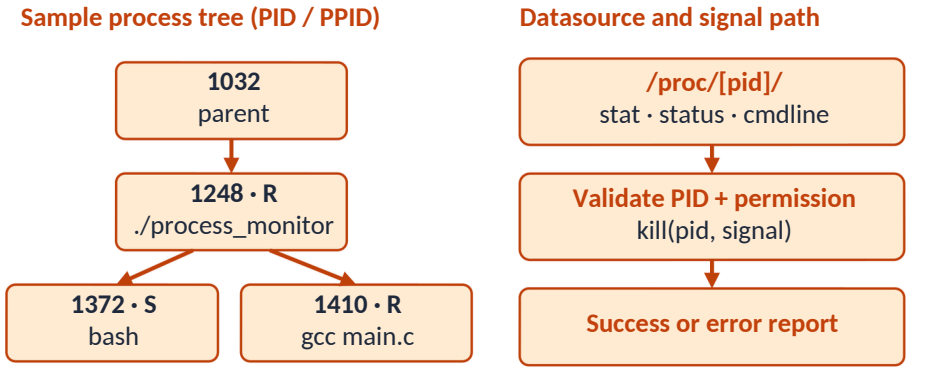
- **Programming:** C /C++

Platform: Linux / Ubuntu
OS Concepts: PID / PPID, process states, signals, /proc filesystem, process scheduling
System Calls / APIs: kill(), POSIX system calls, /proc/[pid]/stat, status, cmdline
Tools: GCC / G++, Git & GitHub

System Workflow / Architecture



Process, IPC & Resource Design



Implementation Screenshots

```
$ ./process_monitor
PID    PPID STATE
1248   1032   R   1372
1248  S  1410  1248  R
signal -> SIGTERM
```

Sample: process list

```
> list
PID    PPID STATE CPU MEM
1248   1032   R        12% 2%
1372 1248 S            2%    1%
> signal 1248 SIGTERM
Signal sent successfully.
```

Sample: signal control

Results & Testing

- **Sample:** signal 1248SIGTERM → “Signal sent successfully.”

Tests: ps / top metric comparison, sample workloads, invalid PIDs, exited processes, permission failures.

Conclusion

- Delivers a lightweight Linux monitor that reads /procand shows PID, PPID, state, CPU, memory and command.

Demonstrates process, scheduling, signaling and resource-monitoring concepts.

Future Scope

- Process-tree view of parent–child relationships and more signals, e.g. stop / continue.

Sorting and filtering by CPU or memory, plus automated tests against ps / top.

References: [1] Linux man-pages: proc(5), kill(2) [2] Operating Systems textbook / course material [3] GCC / GNU documentation